

Universidad Mariano Galvez de Guatemala



Curso: Programación III

Tema: Listas doblemente enlazadas

Docente: David Alvarez

Alumno: Carlos Daniel Castro Caceres

Código corregido:

```
12 #include <iostream> Carlos Daniel Castro Cáceres
13 usando Espacio de nombres STD; // 9941-24-10004
11
9 ~ struct Nodo {
14     int dato;
16     Nodo* siguiente;
202     Nodo* anterior;
205 };
15
203 void insertarInicio(Nodo*& cabeza, Nodo*& cola, int valor) {
207     Nodo* Nuevo = Nuevo Nodo;
204     nuevo->dato = valor;
179     Nuevo->siguiente = cabeza;
206     nuevo->anterior = NULO;
178
181     si (cabeza != NULO) {
208         cabeza->anterior = nuevo;
180     } si no, {
177         cola = nuevo;
182     }
183
1     Cabeza = nuevo;
185 }
184
6 ~ void insertarFinal(Nodo*& cabeza, Nodo*& cola, int valor) {
2     Nodo* nuevo = nuevo Nodo;
3     nuevo->dato = valor;
4     nuevo->siguiente = NULL;
5     Nuevo->Anterior = cola;
```

```
8
10 ~     if (cabeza == NULL) {
7         cabeza = cola = nuevo;
12         Regreso;
13     }
11
9     tail->siguiente = nuevo;
14     cola = nuevo;
16 }
202
15 ~ void mostrarAdelante(cabeza de Nodo* ) {
205     if (cabeza == NULL) {
207         cout << "Lista vacia";
203         Regreso;
204     }
179
206     cout << "\nLista hacia adelante: ";
178     while (cabeza != NULL) {
181         cout << cabeza->dato << " <-> ";
208         cabeza = cara->siguiente;
180     }
177     cout << "NULL\n";
182 }
183
1 ~ void mostrarAtras(Nodo* cola) {
185     if (cola == NULL) {
184         cout << "Lista vacia";
6         Regreso;
```

```

6         Regreso;
2     }
3
4     cout << "\nLista hacia atrás: ";
5     mientras (cola != NULL) {
8         cout << tail->dato << " <-> ";
10        cola = cola->anterior;
7    }
12    cout << "NULL\n";
13 }
11
9     bool buscar(cabeza Nodo*, int valor) {
16     while (cabeza != NULL) {
14         if (head->dato == valor)
202            Retorno verdadero;
15         cabeza = cara->siguiente;
205    }
207    return false;
203 }
204
179 void eliminar(Nodo*& cabeza, Nodo*& cola, int valor) {
206     if (cabeza == NULL) {
178         cout << "Lista vacía";
181         Regreso;
208     }
180
177     Nodo* actual = cabeza;
182
183     mientras que (actual != NULL && actual->dato != valor) {

```

```

183         actual = actual->siguiente;
1      }
185
184     if (actual == NULL) {
6         cout << "Dato no encontrado";
2         Regreso;
3     }
5
4     si (cabeza == cola) {
8         eliminar actual;
10        cabeza = cola = NULL;
7         cout << "Eliminado";
13         Regreso;
12    }
11
14     if (actual == cabeza) {
16         cabeza = cara->siguiente;
9         cabeza->anterior = NULO;
202        eliminar actual;
15         cout << "Eliminado";
205        Regreso;
207    }
203
204     if (actual == cola) {
179        cola = cola->anterior;
206        tail->siguiente = NULL;
178        eliminar actual;
181        cout << "Eliminado";
208        Regreso;

```

```

208         Regreso;
180     }
177
182     actual->anterior->siguiente = actual->siguiente;
183     actual->siguiente->anterior = actual->anterior;
1
185     eliminar actual;
184     cout << "Eliminado";
6 }
2
3 int contarNodos(cabeza Nodo* ) {
5     int contador = 0;
4 while (cabeza != NULL) {
8         Contador++;
10        cabeza = cara->siguiente;
7    }
13    Contador de retorno ;
12 }
11
14 void liberarLista(Nodo*& cabeza, Nodo*& cola) {
16     while (cabeza != NULL) {
9         Nodo* temperatura = cabeza;
202        cabeza = cara->siguiente;
15        eliminar la temperatura;
205    }
207    cola = NULL;
203 }
204
179 Int Main() {

```

```

179 Int Main() {
206     Nodo* cabeza = NULL;
178     Nodo* cola = NULL;
181     int opcion, valor;
208
180 do {
177     Cout << "\n===== MENÚ =====\n";
182     Cout << "1. Insertar al inicio\n";
183     Cout << "2. Insertar al final\n";
1      Cout << "3. Mostrar hacia adelante\n";
185     Cout << "4. Mostrar hacia atrás";
184     Cout << "5. Buscar dato\n";
6      Cout << "6. Eliminar dato\n";
2      Cout << "7. Contar nodos\n";
3      Cout << "8. Salir\n";
5      cout << "Seleccione una opción: ";
4      Cin >> Opcion;
10
8 switch(opcion) {
13     Caso 1:
7         cout << "Ingrese valor: ";
12         Cin >> Valor;
11         insertarInicio(cabeza, cola, valor);
14         Pausa;
16
9     Caso 2:
202        cout << "Ingrese valor: ";
15        Cin >> Valor;
205        insertarFinal(cabeza, cola, valor);

```

```

205     insertarFinal(cabeza, cola, valor);
207     Pausa;
203
204     Caso 3:
179     mostrarAdelante(head);
206     Pausa;
178
208     Caso 4:
181     mostrarAtras (cola);
180     Pausa;
177
182     Caso 5:
183     cout << "Ingrese valor: ";
1     Cin >> Valor;
185     si (buscar(cabeza, valor))
2         cout << "SI existe"";
6         si no,
184         cout << "NO existe"";
3         Pausa;
5
4     Caso 6:
10     cout << "Ingrese valor: ";
8     Cin >> Valor;
13     eliminar(cabeza, cola, valor);
7     Pausa;
12
11     Caso 7:
14     cout << "Total: " << contarNodos(cabeza) << fin;
16     Pausa;

```

```

16         Pausa;
9     }
202
15     } mientras (opcion != 8);
205
207     liberarLista (cabeza, cola);
203     retorno 0;
204 }

```

```
===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 1
Ingrese valor: 10

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 2
Ingrese valor: 20

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 3
```

```
Seleccione una opcion: 3

Lista hacia adelante: 10 <-> 20 <-> NULL

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 4

Lista hacia atras: 20 <-> 10 <-> NULL

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
Seleccione una opcion: 5
Ingrese valor: 20
SI existe

===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
```

```
4. Mostrar hacia atras  
5. Buscar dato  
6. Eliminar dato  
7. Contar nodos  
8. Salir  
Seleccione una opcion: 8
```

```
...Program finished with exit code 0  
Press ENTER to exit console. 
```

Segundo Programa con tail:

```
3  #include <iostream> Carlos Daniel Castro Cáceres
1  usando el espacio de nombres STD; 9941-24-10004
175
2  struct Nodo {
10     int dato;
11     Nodo* siguiente;
13     Nodo* anterior;
12 };
15
16 void insertarInicio(Nodo*& cabeza, Nodo*& cola, int valor) {
199     Nodo* nuevo = nuevo Nodo;
200     nuevo->dato = valor;
14     Nuevo->siguiente = cabeza;
201     Nuevo->anterior = NULL;
200
6     si (cabeza != NULO) {
7         cabeza->anterior = nuevo;
5     } si no, {
4         Cola = Nuevo;
199     }
205
203     Cabeza = nuevo;
202 }
201
205 void insertarFinal(Nodo*& cabeza, Nodo*& cola, int valor) {
204     Nodo* nuevo = nuevo Nodo;
202     nuevo->dato = valor;
203     nuevo->siguiente = NULL;
204     Nuevo->Anterior = cola;
```

```
204     Nodo* nuevo = nuevo Nodo;
202     nuevo->dato = valor;
203     nuevo->siguiente = NULL;
204     Nuevo->Anterior = cola;
8
174     if (cabeza == NULL) {
9         cabeza = cola = nuevo;
3         Regreso;
1     }
2
175     tail->siguiente = nuevo;
10     cola = nuevo;
11 }
13
12 void mostrarAdelante(cabeza de Nodo* ) {
15     if (cabeza == NULL) {
16         cout << "Lista vacia";
199         Regreso;
200     }
14
201     cout << "\nLista hacia adelante: ";
200     while (cabeza != NULL) {
6         cout << cabeza->dato << " <-> ";
7         cabeza = cara->siguiente;
5     }
4     cout << "NULL\n";
199 }
205
203 void mostrarAtras(Nodo* cola) {
```



```

203 void mostrarAtras(Nodo* cola) {
202     if (cola == NULL) {
201         cout << "Lista vacia";
205         Regreso;
204     }
202
203     cout << "\nLista hacia atrás: ";
204     mientras (cola != NULL) {
174         cout << tail->dato << " <-> ";
8         cola = cola->anterior;
3     }
9     cout << "NULL\n";
1     }
2
175 bool buscar(cabeza Nodo*, int valor) {
10     while (cabeza != NULL) {
11         if (head->dato == valor)
13             Retorno verdadero;
12         cabeza = cara->siguiente;
15     }
16     return false;
199 }
200
14 void eliminar(Nodo*& cabeza, Nodo*& cola, int valor) {
201     if (cabeza == NULL) {
200         cout << "Lista vacia";
6         Regreso;
7     }
5

```

```

5     Nodo* actual = cabeza;
4
199     mientras que (actual != NULL && actual->dato != valor) {
205         actual = actual->siguiente;
203     }
202
205     if (actual == NULL) {
201         cout << "Dato no encontrado";
204         Regreso;
202     }
204
203     si (cabeza == cola) {
174         eliminar actual;
8         cabeza = cola = NULL;
3         cout << "Eliminado";
1         Regreso;
9     }
2
175     if (actual == cabeza) {
10         cabeza = cara->siguiente;
13         cabeza->anterior = NULO;
11         eliminar actual;
12         cout << "Eliminado";
15         Regreso;
16     }
199
200     if (actual == cola) {
14         cola = cola->anterior;
201         tail->siguiente = NULL;

```

```

201     tail->siguiente = NULL;
200     eliminar actual;
6       cout << "Eliminado";
7       Regreso;
5     }
4
205     actual->anterior->siguiente = actual->siguiente;
199     actual->siguiente->anterior = actual->anterior;
205
202     eliminar actual;
203     cout << "Eliminado";
201 }
202
204 ~ int contarNodos(cabeza Nodo* ) {
204     int contador = 0;
203 ~     while (cabeza != NULL) {
174         Contador++;
8         cabeza = cara->siguiente;
3     }
9     Contador de retorno ;
1    }
2
175 ~ void liberarLista(Nodo*& cabeza, Nodo*& cola) {
10 ~     while (cabeza != NULL) {
13         Nodo* temperatura = cabeza;
11         cabeza = cara->siguiente;
12         eliminar la temperatura;
15     }
16     cola = NULL;

```

```

16     cola = NULL;
199 }
200
14 ~ Int Main() {
201     Nodo* cabeza = NULL;
200     Nodo* cola = NULL;
6     int opcion, valor;
7
5 ~     do {
205 ~         Cout << "\n===== MENÚ =====\n";
4         Cout << "1. Insertar al inicio\n";
199         Cout << "2. Insertar al final\n";
202         Cout << "3. Mostrar hacia adelante\n";
205         Cout << "4. Mostrar hacia atrás";
203         Cout << "5. Buscar dato\n";
202         Cout << "6. Eliminar dato\n";
201         Cout << "7. Contar nodos\n";
204         Cout << "8. Salir\n";
204         cout << "Seleccione una opción: ";
203         Cin >> Opcion;
174
8 ~         switch(opcion) {
3             Caso 1:
1                 cout << "Ingrese valor: ";
9                 Cin >> Valor;
2                 insertarInicio(cabeza, cola, valor);
10                 Pausa;
175
13             Caso 2:

```

```

13      Caso 2:
11          cout << "Ingrese valor: ";
12          Cin >> Valor;
15          insertarFinal(cabeza, cola, valor);
16          Pausa;
199
200      Caso 3:
14          mostrarAdelante(cabeza);
201          Pausa;
200
206      Caso 4:
7          mostrarAtras (cola);
205          Pausa;
5
4
202      Caso 5:
199          cout << "Ingrese valor: ";
205          Cin >> Valor;
203          si (buscar(cabeza, valor))
202              cout << "SI existe";
201          si no,
204              cout << "NO existe";
204          Pausa;
203
174      Caso 6:
8          cout << "Ingrese valor: ";
3          Cin >> Valor;
1          eliminar(cabeza, cola, valor);
9          Pausa;

```

```

9
2      Caso 7:
10          cout << "Total: " << contarNodos(cabeza) << fin;
175          Pausa;
13      }
11
16  } mientras (opcion != 8);
15
12  liberarLista (cabeza, cola);
200  retorno 0;
199 }

```

```
===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
```

Seleccione una opcion: 1
Ingrese valor: 10

```
===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
```

Seleccione una opcion: 2
Ingrese valor: 15

```
===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
```

```
7. Contar nodos
8. Salir
```

Seleccione una opcion: 3

Lista hacia adelante: 10 <-> 15 <-> NULL

```
===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
```

Seleccione una opcion: 4

Lista hacia atras: 15 <-> 10 <-> NULL

```
===== MENU =====
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
```

Seleccione una opcion: 5
Ingrese valor: 3
NO existe

```
===== MENU =====
```

```
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
```

Seleccione una opcion: 6

Ingrese valor: 15

Eliminado

===== MENU =====

```
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
```

Seleccione una opcion: 7

Total: 1

===== MENU =====

```
1. Insertar al inicio
2. Insertar al final
3. Mostrar hacia adelante
4. Mostrar hacia atras
5. Buscar dato
6. Eliminar dato
7. Contar nodos
8. Salir
```

Seleccione una opcion: